

EXPERIMENT NO: 1. Write code for a simple user registration form for an event.

Aim: Write code for a simple user registration form for an event.

```
<div class="container">
  <h1>Event Registration Form</h1>
  <form name="Registration" class="registration-form" onsubmit="return formValidation()">
<table>
<tr>
  <td><label for="name">Name:</label></td>
  <td><input type="text" name="name" id="name" placeholder="yourname"></td>
</tr>
<tr>
  <td><label for="email">Email:</label></td>
  <td><input type="text" name="email" id="email" placeholder="youremail"></td>
</tr>
<tr>
  <td><label for="password">Password:</label></td>
  <td><input type="password" name="password" id="password"></td>
</tr>
<tr>
  <td><label for="phoneNumber"> Phone Number:</label></td>
  <td><input type="number" name="phoneNumber" id="phoneNumber"></td>
</tr>
<tr>
  <td><label for="gender">Gender:</label></td>
  <td>Male: <input type="radio" name="gender" value="male"> Female: <input type="radio" name="gender" value="female">
    Other: <input type="radio" name="gender" value="other"></td>
</tr>
<tr>
  <td><label for="language">language</label></td>
  <td><select name="language" id="language">
    <option value="English">English
    <option value="Hindi">Hindi
    <option value="Tamil">Tamil
    <option value="Telugu">Telugu
    <option value="Kannada">Kannada
    <option value="Malayalam">Malayalam
    <option value="Marathi">Marathi
    <option value="Gujarati">Gujarati
    <option value="Bengali">Bengali
    <option value="Punjabi">Punjabi
    <option value="Urdu">Urdu
    <option value="Sinhala">Sinhala
    <option value="Tamil">Tamil
    <option value="Telugu">Telugu
    <option value="Kannada">Kannada
    <option value="Malayalam">Malayalam
    <option value="Marathi">Marathi
    <option value="Gujarati">Gujarati
    <option value="Bengali">Bengali
    <option value="Punjabi">Punjabi
    <option value="Urdu">Urdu
    <option value="Sinhala">Sinhala
  </select></td>
</tr>
</table>
</div>
```

```

        <option value="">Select language</option>
        <option value="English">English</option>
        <optionvalue="Spanish">Spanish</option>
        <optionvalue="Hindi">Hindi</option>
        <optionvalue="Arabic">Arabic</option>
        <optionvalue="Russian">Russian</option>
    </select>
</td>
</tr>
<tr>
    <td><label for="zipcode">ZipCode:</label></td>
    <td><input type="number" name="zipcode" id="zipcode"></td>
</tr>
<tr>
    <td><label for="about">About:</label></td>
    <td><textarea name="about" id="about" placeholder="Writeaboutyourself..."
></textarea></td>
</tr>
<tr>
    <td colspan="2"><input type="submit" class="submit" value="Register"/></td>
</tr></table>
</form>
</div>

```

Event Registration Form

Name:	your name
Email:	your email
Password:	
Phone Number:	
Gender:	Male: ☐ Female: ☐ Other: ☐
language	Select language
Zip Code:	
About:	Write about yourself...
Register	

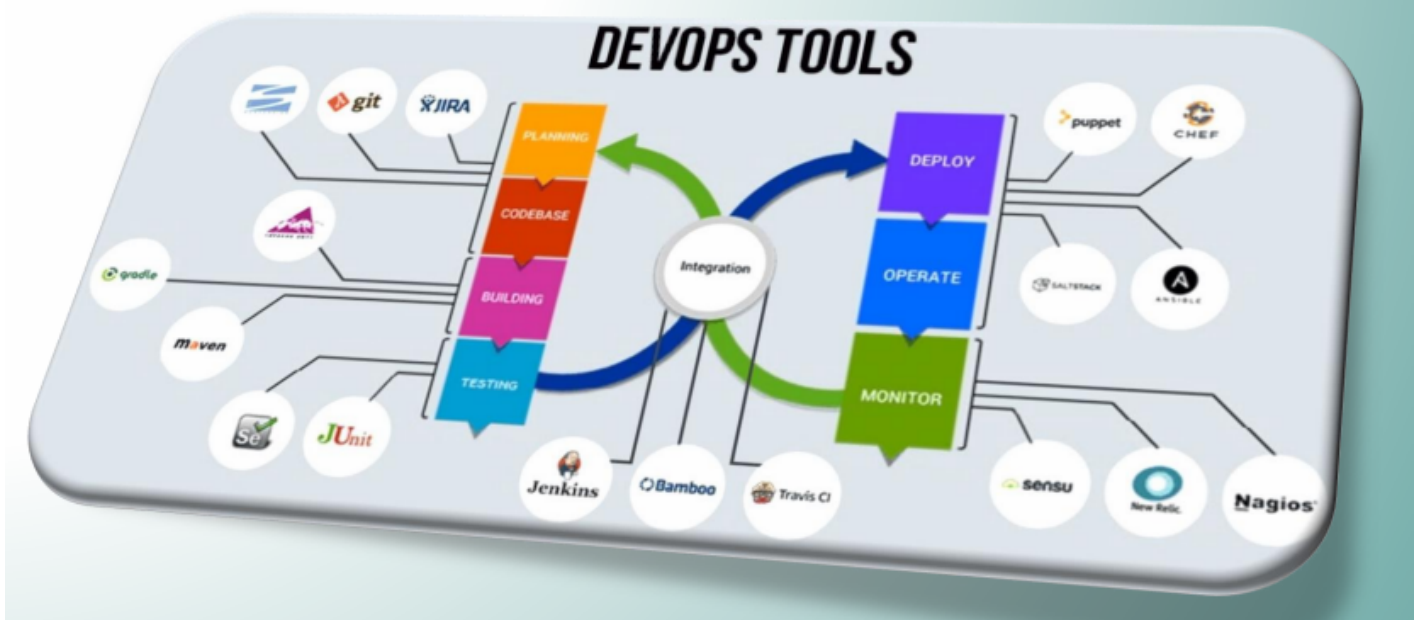
EXPERIMENTNO:2. Explore Git and GitHub Commands

Aim: Explore Git and GitHub commands

DESCRIPTION:

Git and GitHub are two of the most popular tools used for version control and collaboration in software development.

Version Control [Git]



Version control is a system that records/manages changes to documents, computer programs etc over time. It helps us tracking changes when multiple people work on the same project.

Advantages of Version Control:

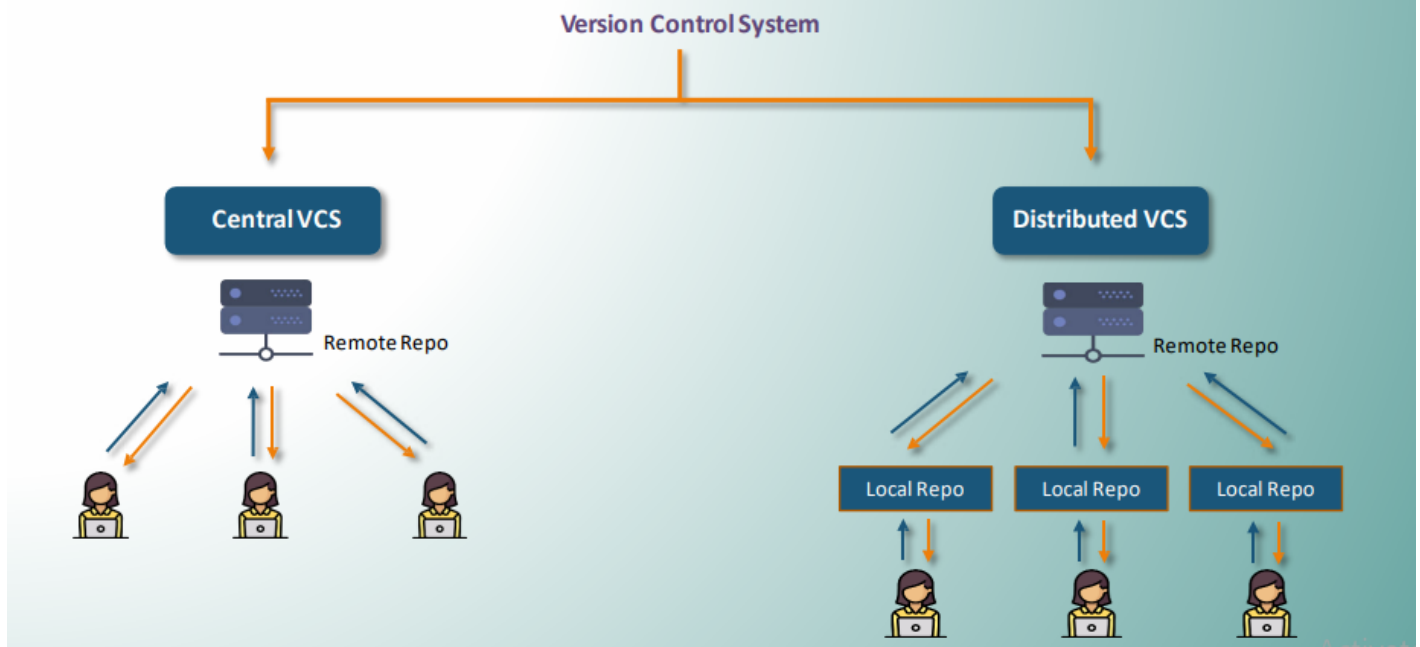
Versioning is Automatic.

Team Collaboration is simple.

Easy Access to previous Versions.

Only modified code is stored across different versions, hence saves storage.

TYPES OF VERSION CONTROL SYSTEM



Git is the most popular tool among all the DVCS tools. It is a version-control system for tracking changes in computer files and coordinating work on those files among multiple people. It is primarily used for source-code management in software development, but it can be used to keep track of changes in any set of files.

GIT LIFECYCLE

Following are the lifecycle stages of files in Git



Working Directory:

The place where your project resides in your local disk

This project may or may not be tracked by git. In either case, the directory is called the working directory.

The project can be tracked by git, by using the command `git init`.
By doing `git init`, it automatically creates a hidden `.git` folder

Staging Area:

Once we are in the working directory, we have to specify which files are to be tracked by git. We do not specify all files to be tracked in git, because some files could be temporary data which is being generated while execution. To add files in the staging area, we use the command `git add`.

Commit:

Once the files are selected and are ready in the staging area, they can now be saved in repository. Saving a file in the repository of git is known as doing a commit. When we commit a repository in git, the commit is identified by a commit id.

The command for initializing this process is `git commit -m "message"`

Here are some common Git and GitHub commands:

Initializing a Git repository: You can create a repository using the command **git init**. Navigate to your project folder and enter the command `git init` to initialize a git repository for your project on the local system.

git init

```
ubuntu@ip-172-31-33-5:~/project$ ls
1.txt  2.txt
ubuntu@ip-172-31-33-5:~/project$ git init
Initialized empty Git repository in /home/ubuntu/project/.git/
ubuntu@ip-172-31-33-5:~/project$
```

Git status: Once the directory has been initialized you can check the status of the files, whether they are being tracked by git or not, using the command.

git status

```
ubuntu@ip-172-31-33-5:~/project$ ls
1.txt  2.txt
ubuntu@ip-172-31-33-5:~/project$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        1.txt
        2.txt

nothing added to commit but untracked files present (use "git add" to track)
ubuntu@ip-172-31-33-5:~/project$
```

Adding all files to the stage: Since no files are being tracked right now, let us now stage these files. For that, enter the command `git add .`. If we want to track all the files in the project folder, we can type the command, **git add .**

```

ubuntu@ip-172-31-33-5:~/project$ ls
1.txt  2.txt
ubuntu@ip-172-31-33-5:~/project$ git add .
ubuntu@ip-172-31-33-5:~/project$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   1.txt
        new file:   2.txt

ubuntu@ip-172-31-33-5:~/project$ █

```

Adding a file to the stage: A single file can be added to the staging area using the following command.
git add <file-name>

Committing changes: Once the files or changes have been staged, we are ready to commit them in our repository. We can commit the files using the command **git commit -m "custom message"**

```

ubuntu@ip-172-31-33-5:~/project$ ls
1.txt  2.txt
ubuntu@ip-172-31-33-5:~/project$ git commit -m "First Commit"

2 files changed, 2 insertions(+)
create mode 100644 1.txt
create mode 100644 2.txt

```

Pushing Changes to a Remote repository: Once everything is ready on our local, we can start pushing our changes to the remote repository. Copy your repository link and paste it in the command
git remote add origin "URL to Repository"

```

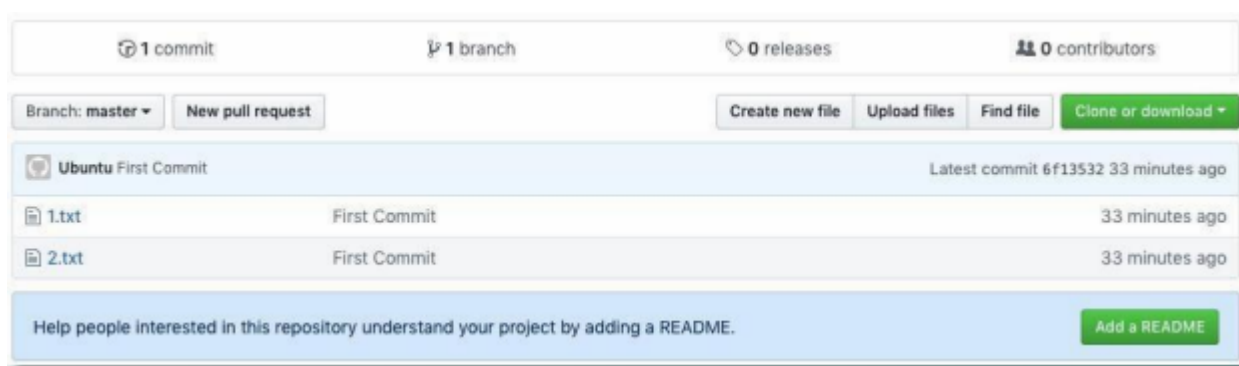
ubuntu@ip-172-31-33-5:~/project$ git remote add origin "https://github.com/devops/devops.git"
ubuntu@ip-172-31-33-5:~/project$ █

```

Pushing changes to a remote repository: To push the changes to your repository, enter the command **git push origin** and hit enter. In our case the branch is master, hence the command is **git push origin master**. This command will then prompt for username and password, enter the values and hit enter.

```
ubuntu@ip-172-31-33-5:~/project$ git push origin master
Username for 'https://github.com': devops
Password for 'https://devops@github.com'
Counting objects: 4, done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 292 bytes | 292.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/devops/devops/pull/new/master
remote:
To https://github.com/devops/devops.git
 * [new branch]      master -> master
ubuntu@ip-172-31-33-5:~/project$
```

Your local repository is now synced with the remote repository on github.



Cloning a repository from GitHub : If we want to download the remote repository to our local system, we can use the command: **git clone**. This command will create a folder with the repository name, and download all the contents of the repository inside this folder. In our example, repository contents were downloaded into the "devops" folder.

```
ubuntu@ip-172-31-33-5:~$ git clone https://github.com/devops/devops.git
Cloning into 'devops'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), done.
ubuntu@ip-172-31-33-5:~$ ls
devops  project
ubuntu@ip-172-31-33-5:~$
```

Git Pull: The git pull command is also used for pulling the latest changes from the repository, unlike git clone, this command can only work inside an initialized git repository. This command is used when you are already working in the cloned repository, and want to pull the latest changes, that others might have pushed to the remote repository **git pull**

```
ubuntu@ip-172-31-33-5:~/devops$ git pull https://github.com/devops/devops.git:
From https://github.com/devops/devops
 * branch                HEAD      -> FETCH_HEAD
Already up to date.
ubuntu@ip-172-31-33-5:~/devops$
```

Creating a new branch: When Multiple developers working on the same project or repository, To handle the workspace of multiple developers, we use branches. To create a branch from an existing branch, we type **git branch <new branch name>** Similarly, to delete a branch use the command **git branch -D <branch-name>**

```
ubuntu@ip-172-31-33-5:~$ cd devops
ubuntu@ip-172-31-33-5:~/devops$ git branch branch1
ubuntu@ip-172-31-33-5:~/devops$
```

Switching to a different branch: git checkout is used in Git to switch branches or restore working tree files. Depending on how you use it, git checkout can serve several purposes:

Switching Branches:

To switch to an existing branch: **git checkout <branch-name>**

To create and switch to a new branch: **git checkout -b <new-branch-name>**

Checking Out Specific Files:

To discard changes in a specific file: **git checkout -- <file>**

To revert a file to a specific commit: **git checkout <commit> -- <file>**

Merging two branches: git merge is a Git command used to incorporate changes from one branch into another branch. It combines the changes made on two branches and creates a new commit that represents the combination of both branches' histories.

Here's how you typically use '**git merge**':

Merge a Branch into the Current Branch:

First, ensure you are on the branch where you want to merge changes (**git checkout <branch>**).

Then, merge another branch into the current branch: **git merge <other-branch>**.

This command will integrate the changes from <other-branch> into the current branch.

EXPERIMENTNO:3. Practice Source code management on GitHub. Experiment with the source code written in exercise 1

Aim: To Practice Source code management on GitHub.

Description:

To practice source code management on GitHub, you can follow these steps:

- Create a GitHub account.
- Create a new repository on GitHub.
- Clone the repository to your local machine: `$ git clone <repository-url>`
- Move to the repository directory: `$ cd <repository-name>`
- Create a new file in the repository and add the source code written in exercise 1.
- Stage the changes: `$ git add <file-name>`
- Commit the changes: `$ git commit -m "Added source code for a simple user registration form"`
- Push the changes to the remote repository: `$ git push origin master`
- Verify that the changes are reflected in the repository on GitHub.

These steps demonstrate how to use GitHub for source code management.

You can use the same steps to manage any source code projects on GitHub. Additionally, you can also explore GitHub features such as pull requests, code review, and branch management to enhance your source code management workflow.

ADD Screenshot of registration file pushed to GitHub here

EXPERIMENTNO:4. Jenkins installation and setup, explore the environment

Aim: Jenkins installation and setup, explore the environment

DESCRIPTION

Jenkins is a popular open-source tool for Continuous Integration and Continuous Deployment (CI/CD) in software development.

- Jenkins is an open-source automation tool written in Java programming language.
- Jenkins is free and allows continuous integration.
- Continuous integration
 - Continuous integration refers to the build and unit testing stages of the software release process. Every revision that is committed triggers an automated build and test. With continuous delivery, code changes are automatically built, tested, and prepared for a release to production.
- Jenkins is a server-based application and requires a web server like Apache Tomcat.
- Jenkins builds and tests our software projects continuously.
 - This is the main reason for Jenkins to become so popular, continuously monitoring of repeated tasks which arise during the development of a project.
 - Example, if your team is developing a project, Jenkins will continuously test your project builds and show you the errors in early stages of your development.
- Benefits of using Jenkins CI
 - **Reduced Development Cycle** – Since every commit is built and tested, it allows releasing new features to the user faster and with fewer errors.
 - **Shorter Time to Integrate Code**– Before the use of Jenkins CI, integration of code was done manually, thus taking a few days. In some cases, it might happen that the code is not running, and it is hard to debug as it might have gone through various commits in the repository. Integrating code after every commit ensures that the functionality is not broken after a commit.
 - **Faster Feedback Loops** – Developers get feedback and improve the code whenever test breaks during a commit. Otherwise, debugging the issue can be very difficult, given teams would not be sure which commit resulted in the bug.
 - **Automated Workflow** – Teams should not worry about running a manual test for each commit. The Jenkins CI pipeline checks the latest code and builds the code along with the tests. The test can deploy the project in a specific environment if it is green. It can notify the developer by breaking the build.

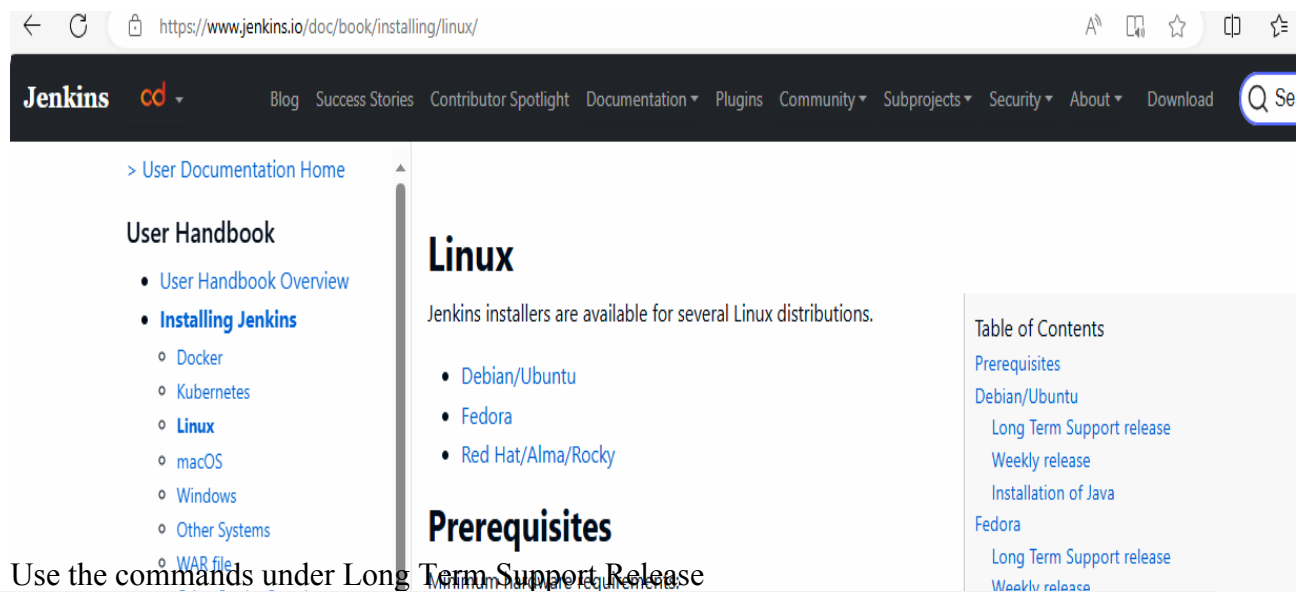
Here are the steps to install and setup Jenkins:

Download and install Jenkins:

- Download the Jenkins package for your operating system from the Jenkins website.
- To download and install Jenkins on Ubuntu, you can follow these steps. Jenkins is typically installed on Ubuntu using the package manager and requires Java to be installed as a prerequisite.

- **Step 1: Install Java**
- First, you need to install Java Development Kit (JDK) because Jenkins is a Java-based application.
- Install Java using the following command.

```
sudo apt update
sudo apt install fontconfig openjdk-17-jre
```



- Use the commands under Long Term Support Release


```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins
```

Start Jenkins

You can enable the Jenkins service to start at boot with the command:

```
sudo systemctl enable jenkins
```

You can start the Jenkins service with the command:

```
sudo systemctl start jenkins
```

You can check the status of the Jenkins service using the command:

```
sudo systemctl status Jenkins.
```

Post-installation setup wizard:

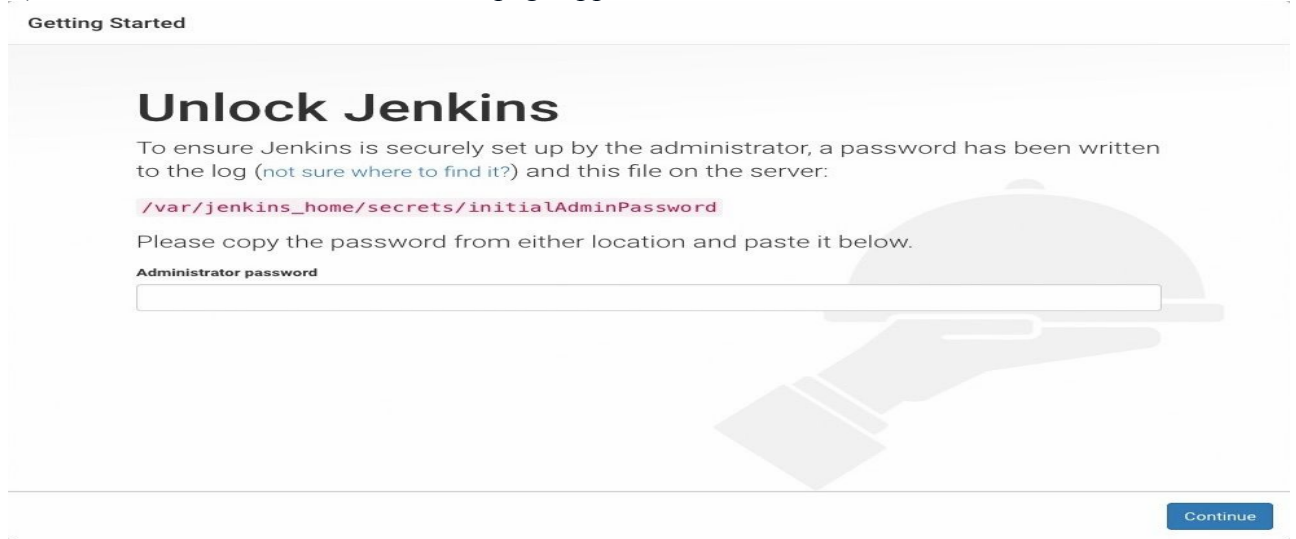
After downloading, installing and running Jenkins using one of the procedures above (except for installation with Jenkins Operator), the post-installation setup wizard begins.

This setup wizard takes you through a few quick "one-off" steps to unlock Jenkins, customize it with plugins and create the first administrator user through which you can continue accessing Jenkins.

Unlocking Jenkins

When you first access a new Jenkins instance, you are asked to unlock it using an automatically-generated password.

1. Browse to `http://localhost:8080` (or whichever port you configured for Jenkins when installing it) and wait until the **Unlock Jenkins** page appears.



Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`/var/jenkins_home/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

- If the Jenkins service is running, you will see the Jenkins login page.

Initialize the Jenkins environment:

- Follow the instructions on the Jenkins setup wizard to initialize the Jenkins environment.
- This process involves installing recommended plugins, setting up security, and creating the first admin user.

Explore the Jenkins environment:

- Once the Jenkins environment is setup, you can explore the various features and functionalities available in the web interface.
- Jenkins has a rich user interface that provides access to features such as build history, build statistics, and system information.

These are the basic steps to install and set up Jenkins. Depending on your usecase, you may need to customize your Jenkins environment further. For example, you may need to configure build agents, set up build pipelines, or integrate with other tools. However, these steps should give you a good starting point for using Jenkins for CI/CD in your software development projects.

EXPERIMENTNO:5. Demonstrate continuous integration and development using Jenkins.

Aim: Demonstrate continuous integration and development using Jenkins.

DESCRIPTION:

Continuous Integration (CI) and Continuous Development (CD) are important practices in software development that can be achieved using Jenkins. Here's an example of how you can demonstrate CI/CD using Jenkins:

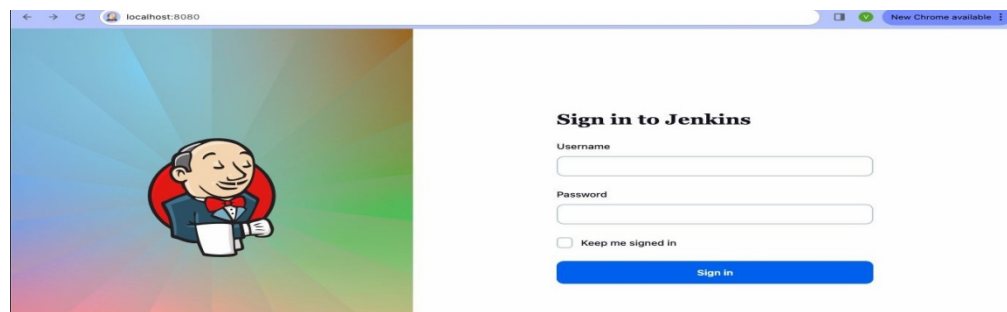
Create a simple Java application that you want to integrate with Jenkins.

The application should have some basic functionality, such as printing "Hello World" or performing simple calculations.

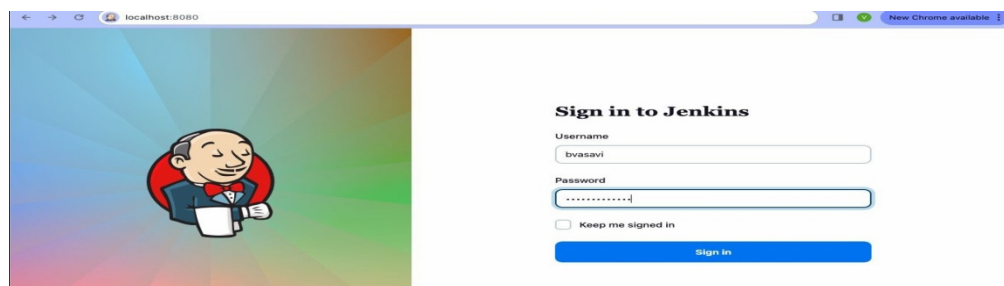
Commit the code to a Git repository: Create a Git repository for the application and commit the code to the repository.

Make sure that the Git repository is accessible from the Jenkins server.

Open Jenkins tool by typing localhost:8080 in a web browser

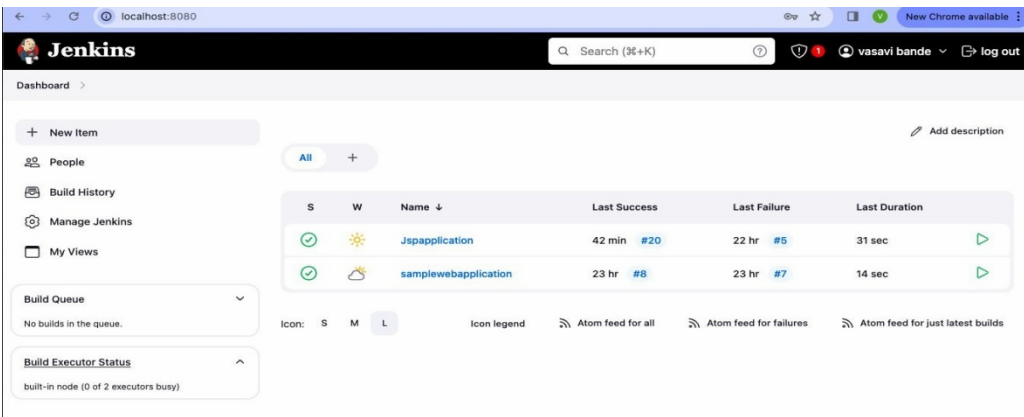


Login to your Jenkins server



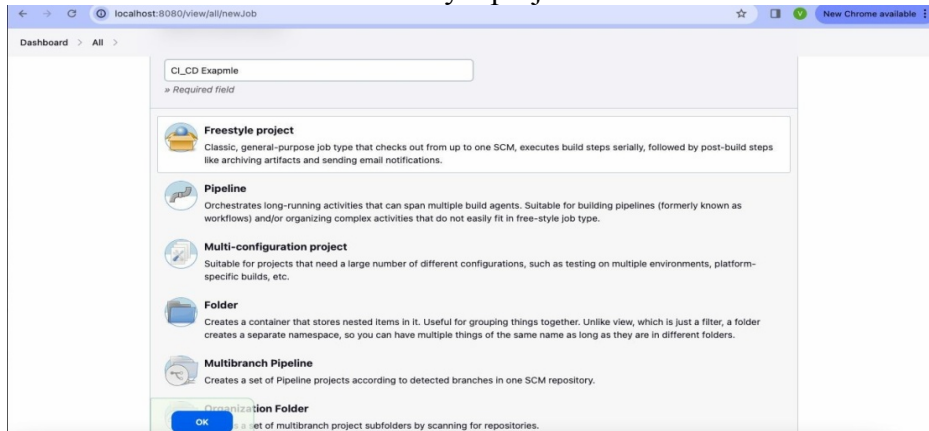
Jenkins page

Create a Jenkins job: Log in to the Jenkins web interface and create a new job.



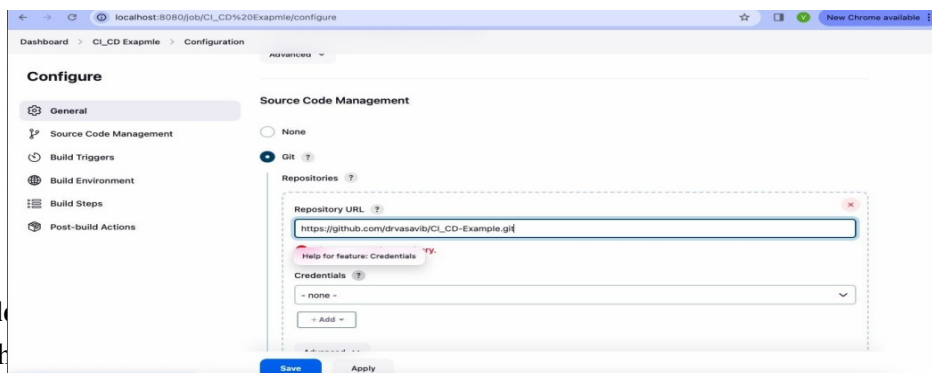
Click on Newitem

- Give a name and select freestyle project and then ok.



Configure the job to build the Java application from the Git repository.

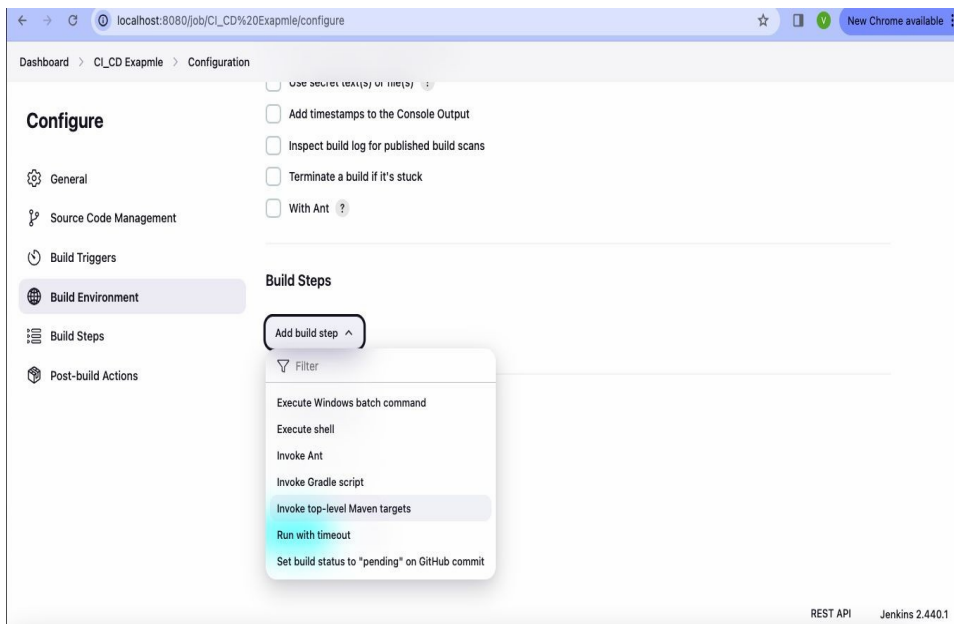
- Select Git under source code management
- Give your git repository (CI_CD Example) URL in the Repository URL
- Click on save.



Build

- The

Scroll down and goto Build steps and select **Execute shell**



Give shell commands as `javac filename.java`
Java filename

Click on **Buildnow**. Check the output in console output.

Monitor the build: Monitor the build progress in the Jenkins web interface.

- The build should show the build log, test results, and the status of the build.

Deploy the application: If the build is successful, configure the Jenkins job to deploy the application to a production environment.

The deployment could be as simple as copying the jar file to a production server or using a more sophisticated deployment process, such as using a containerization technology like Docker. Jenkins should automatically build and deploy the changes to the production environment. This is a basic example of how you can use Jenkins to demonstrate CI/CD in software development.

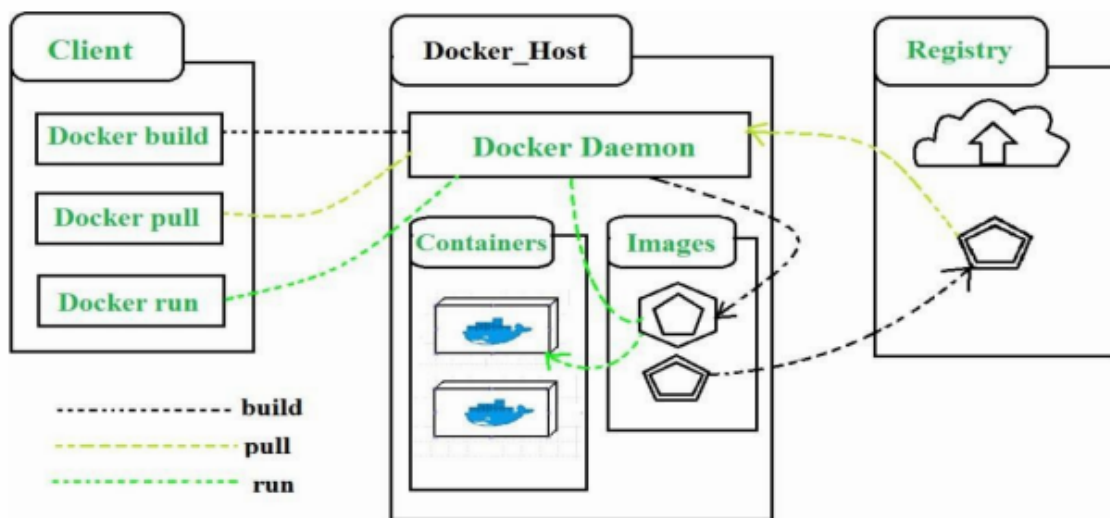
EXPERIMENT NO.: 6. Explore Docker commands for content management.

AIM: Explore Docker commands for content management.

DESCRIPTION: Docker is a containerization technology that is widely used for managing application containers.

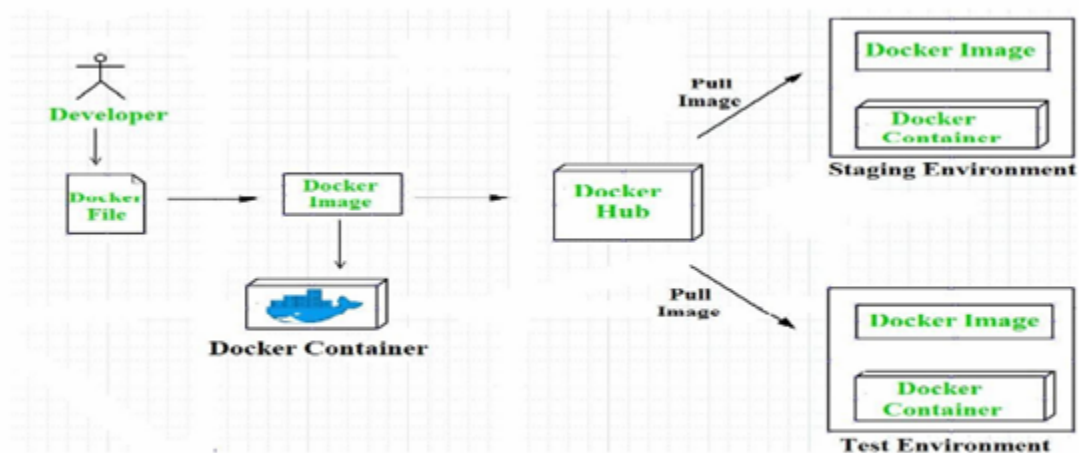
Docker Architecture:

Docker architecture consists of Docker client, Docker Daemon running on Docker Host and DockerHub repository.



Components of Docker

The main components of Docker include—Docker clients and servers, Docker images, Docker file, Docker Registries, and Docker containers.



Here are some commonly used Docker commands for content management:

Docker run: Run a command in a new container.

- For example: `$ sudo docker run --name mycontainer -it ubuntu:16.04 /bin/bash`

This command runs a new container based on the Ubuntu 16.04 image and starts a shell session in the container.

Docker start: Start one or more stopped containers.

- For example: `$ sudo docker start mycontainer`

This command starts the container named "mycontainer".

Docker stop: Stop one or more running containers.

- For example: `$ sudo docker stop mycontainer`

This command stops the container named "mycontainer".

Docker rm: Remove one or more containers.

- For example: `$ sudo docker rm mycontainer`

This command removes the container named "mycontainer".

Docker ps: List containers.

- For example: `$ docker ps`

This command lists all running containers.

Docker images: List images.

- For example: `$ docker images`

This command lists all images stored locally on the host.

Docker pull: Pull an image or a repository from a registry.

- For example: `$ docker pull ubuntu:16.04`

This command pulls the Ubuntu 16.04 image from the Docker Hub registry.

Docker push: Push an image or a repository to a registry.

- For example: `$ docker push myimage`

This command pushes the image named "myimage" to the Docker Hub registry.

These are some of the basic Docker commands for managing containers and images. There are many other Docker commands and options that you can use for more advanced use cases, such as managing networks, volumes, and configuration.

EXPERIMENT NO.: 7. Develop a simple containerized application using Docker

AIM: Develop a simple containerized application using Docker

DESCRIPTION Here's an example of how you can develop a simple containerized application using Docker

Choose an application: Choose a simple application that you want to containerize. For example, a Python script that prints "Hello World".

Write a Dockerfile: Create a file named "Dockerfile" in the same directory as the application. In the Dockerfile, specify the base image, copy the application into the container, and specify the command to run the application.

Here's an example Dockerfile for a Python script: # Use the official Python image as the base image

```
FROM python:3.9 # Copy the Python script into the container
```

```
COPY hello.py /app
```

```
# Set the working directory to /app/
```

```
WORKDIR /app/
```

```
# Run the Python script when the container starts
```

```
CMD ['python' 'hello.py']
```

• Build the Docker image:

Run the following command to build the Docker image:

```
$ sudo docker build -t myimage .
```

This command builds a new Docker image using the Dockerfile and tags the image with the Name 'myimage'.

• Run the Docker container:

Run the following command to start a new container based on the image:

```
$ sudo docker run --name mycontainer myimage
```

This command starts a new container named "mycontainer" based on the "myimage" image and runs the Python script inside the container.

• Verify the output:

Run the following command to verify the output of the container:

```
$ sudo docker logs mycontainer
```

This command displays the logs of the container and should show the "Hello World" output.

This is a simple example of how you can use Docker to containerize an application. In a real-world scenario, you would likely have more complex requirements, such as running multiple containers, managing network connections, and persisting data.

Example 2: A simple user registration form using Flask and Docker in DevOps.

Create a Docker file with the following content to create a Docker image for your Flask application:

Dockerfile

```
FROM python:3.8
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

- Create a **requirements.txt** file with the following content to list the dependencies of your Flask application:

```
Flask==1.1.2
```

- Create a **app.py** file with the following code for a simple user registration form in Flask:

```
from flask import Flask, request, render_template
app = Flask( name )
@app.route(register.methods=[GET, POST])
def register():
    if request.method == POST:
        name = request.form[name]
        email = request.form[email]
        password = request.form[password]
        # Store the user data in a database or file return render_template(success.html)

    return render_template(register.html)
if __name__ == '__main__': app.run(host=0.0.0.0)
```

- Create a templates folder and add the following two files: register.html and success.html.

-

register.html

```
<form method=post>
<input type="text" name="name" placeholder="Name">
<input type="email" name="email" placeholder="Email">
<input type="password" name="password" placeholder="Password">
```

```
<input type="submit" value="Submit"> </form>
```

success.html

```
<h2>Registration Successful </h2>
```

- Build the Docker image for your Flask application using the following command:
 docker build -t simple-flask-app
- Run a Docker container from the image using the following command:

- docker run -p 5000:5000 simple-flask-app
- Open a web browser and access the registration form at
http://localhost:5000/register.

•

This example demonstrates how to build a simple user registration form in Flask and run it in a Docker container in DevOps.

EXPERIMENT NO.: 8. Integrate Kubernetes and Docker

AIM: Integrate Kubernetes and Docker

DESCRIPTION:

Kubernetes and Docker are both popular technologies for managing containers, but they are used for different purposes. Kubernetes is an orchestration platform that provides higher-level abstractions for managing containers, while Docker is a containerization technology that provides a lower-level runtime for containers.

To integrate Kubernetes and Docker, you need to use Docker to build and package your application as a container image, and then use Kubernetes to manage and orchestrate the containers. Here's a high-level overview of the steps to integrate Kubernetes and Docker:

- **Build a Docker image:**

Use Docker to build a Docker image of your application. You can use a Dockerfile to specify the base image, copy the application into the container, and specify the command to run the application.

- **Push the Docker image to a registry:**

Push the Docker image to a container registry, such as Docker Hub or Google Container Registry, so that it can be easily accessed by Kubernetes. Deploy the Docker image to a Kubernetes cluster: Use Kubernetes to deploy the Docker image to a cluster.

This involves creating a deployment that specifies the number of replicas and the image to be used, and creating a service that exposes the deployment to the network.

- **Monitor and manage the containers:** Use Kubernetes to monitor and manage the containers. This includes scaling the number of replicas, updating the image, and rolling out updates to the containers.

- **Continuously integrate and deploy changes:**

Use a continuous integration and deployment (CI/CD) pipeline to automatically build, push, and deploy changes to the Docker image and the Kubernetes cluster. This makes it easier to make updates to the application and ensures that the latest version is always running in the cluster.

By integrating Kubernetes and Docker, you can leverage the strengths of both technologies to manage containers in a scalable, reliable, and efficient manner

EXPERIMENT NO.: 9. Automate the process of running containerized application developed in exercise 7 using Kubernetes

AIM: Automate the process of running containerized application developed in exercise 7 using Kubernetes

DESCRIPTION To automate the process of running the containerized application developed in exercise 7 using Kubernetes, follow the below mentioned steps:

Create a Kubernetes cluster: Create a Kubernetes cluster using a local installation of Minikube.

Push the Docker image to a registry:

Push the Docker image of your application to a container registry, such as Docker Hub or Google Container Registry.

Create a deployment: Create a deployment in Kubernetes that specifies the number of replicas and the Docker image to use. Here's an example of a deployment YAML file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    match Labels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myimage
          ports:
            - containerPort: 80
```

- **Create a service:**
Create a service in Kubernetes that exposes the deployment to the network. Here's an example of a service YAML file:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
    - name: http
      port: 80
      targetPort: 80
  type: ClusterIP
```

- **Apply the deployment and service to the cluster:**
Apply the deployment and service to the cluster using the kubectl command line tool.
For example: `$ kubectl apply -f deployment.yaml`

```
$ kubectl apply -f service.yaml
```

- **Verify the deployment:** Verify the deployment by checking the status of the pods and the service.
- For example: `$ kubectl get pods`

```
$ kubectl get services
```

This is a basic example of how to automate the process of running a containerized application using Kubernetes. In a real-world scenario, you would likely have more complex requirements, such as managing persistent data, scaling, and rolling updates, but this example should give you a good starting point for using Kubernetes to manage your containers.

EXPERIMENT NO.: 10. Install and Explore Selenium for automated testing

AIM: Install and Explore Selenium for automated testing

DESCRIPTION: To install and explore Selenium for automated testing, you can follow these steps: Install Java Development Kit (JDK):

- Selenium is written in Java, so you'll need to install JDK in order to run it.
- You can download and install JDK from the official Oracle website. Install the Selenium WebDriver:
- You can download the latest version of the Selenium WebDriver from the Selenium website.
- You'll also need to download the appropriate driver for your web browser of choice (e.g. Chrome Driver for Google Chrome).
- Install an Integrated Development Environment (IDE): To write and run Selenium tests, you'll need an IDE. Some popular choices include Eclipse, IntelliJ IDEA, and Visual Studio Code
- Write a simple test:
- Once you have your IDE set up, you can write a simple test using the Selenium WebDriver.

Here's an example in Java:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class Main
{
    public static void main(String[] args)
    {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        WebDriver driver = new ChromeDriver();
        driver.get("https://www.google.com");
        System.out.println(driver.getTitle());
        driver.quit();
    }
}
```

Run the test: Run the test using your IDE or from the command line using the following command:

```
$ javac Main.java
$ java Main
```

This is a basic example of how to get started with Selenium for automated testing. In a real-world scenario, you would likely write more complex tests and organize your code into test suites and test cases, but this example should give you a good starting point for exploring Selenium.

EXPERIMENT NO: 11. Write a simple program in JavaScript and perform testing using Selenium

AIM: Write a simple program in JavaScript and perform testing using Selenium

Simple JavaScript program that you can test using Selenium

PROGRAM: Simple JavaScript program that you can test using Selenium

```
<!DOCTYPE html>
<html>
<head>
  <title>Simple JavaScript Program</title>
</head>
<body>
  <p id="output">0</p>
  <button id="increment-button">Increment</button>
  <script>
    const output = document.getElementById("output");
    const incrementButton=document.getElementById("increment-button");

    let count = 0;

    incrementButton.addEventListener("click", function() {
      count += 1;
      output.innerHTML = count;
    });
  </script>
</body>
</html>
```

- Write a test case for this program using Selenium

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;import
org.junit.After;
import org.junit.Before;import
org.junit.Test;
```

```
public class Main {
  private WebDriver driver;
```

```

@Before
public void setUp() {
    System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");driver = new
    ChromeDriver();
}

@Test
public void testIncrementButton()
{ driver.get("file:///path/to/program.html");
  driver.findElement(By.id("increment-button")).click();
  String result = driver.findElement(By.id("output")).getText();
  assert result.equals("1");
}

@After
public void tearDown()
{
  driver.quit();
}
}

```

You can run the test case using the following command:

```
$ javac Main.java
```

```
$ java Main
```

The output of the test case should be:

```
.
```

```
Time: 0.189
```

```
OK (1 test)
```

This output indicates that the test case passed, and the increment button was successfully clicked, causing the output to be incremented by 1.